

Design and V&V Document

NexDerm - Version 0

Group 11

COMPSCI 4ZP6

Dr. Mehdi Moradi

January 23rd, 2026

Name	Email	Student Number
Divye Goswami	goswad3@mcmaster.ca	400362533
Tanvir Singh	singht48@mcmaster.ca	400378756
Hetanshu Pandya	pandyh6@mcmaster.ca	400371731
Ishaan Jamwal	jamwali@mcmaster.ca	400389636
Harkunwar Singh	singh42@mcmaster.ca	400380573

Table of Contents

Table of Contents.....	2
Design Document.....	3
1. Team Members.....	3
2. Version.....	3
3. Purpose statement for the project.....	3
4. Component diagram:.....	3
5. Relationship Between Project Components and Requirements:.....	4
6. Normal Behavior, API, Implementation, Undesired Behavior:.....	5
6.1 User Interface (UI) / Frontend (Mobile Application).....	5
6.2 Image Processing.....	6
6.3 Database Subsystem.....	6
6.4 AI Model Subsystem with Backend logic.....	7
6.5 Training/Testing Module.....	8
7. Details on User Interface:.....	9
Validation and Verification.....	11
1. User Interface (UI) / Frontend (Mobile Application).....	11
2 Image Processor.....	11
3 Database Subsystem.....	12
4 AI Model Subsystem.....	12
5 Training/Testing Module.....	13

Design Document

1. Team Members

Divye Goswami: goswad3@mcmaster.ca, 400362533
Tanvir Singh: singht48@mcmaster.ca, 400378756
Hetanshu Pandya: pandyh6@mcmaster.ca, 400371731
Ishaan Jamwal: jamwali@mcmaster.ca, 400389636
Harkunwar Singh: singh42@mcmaster.ca, 400380573

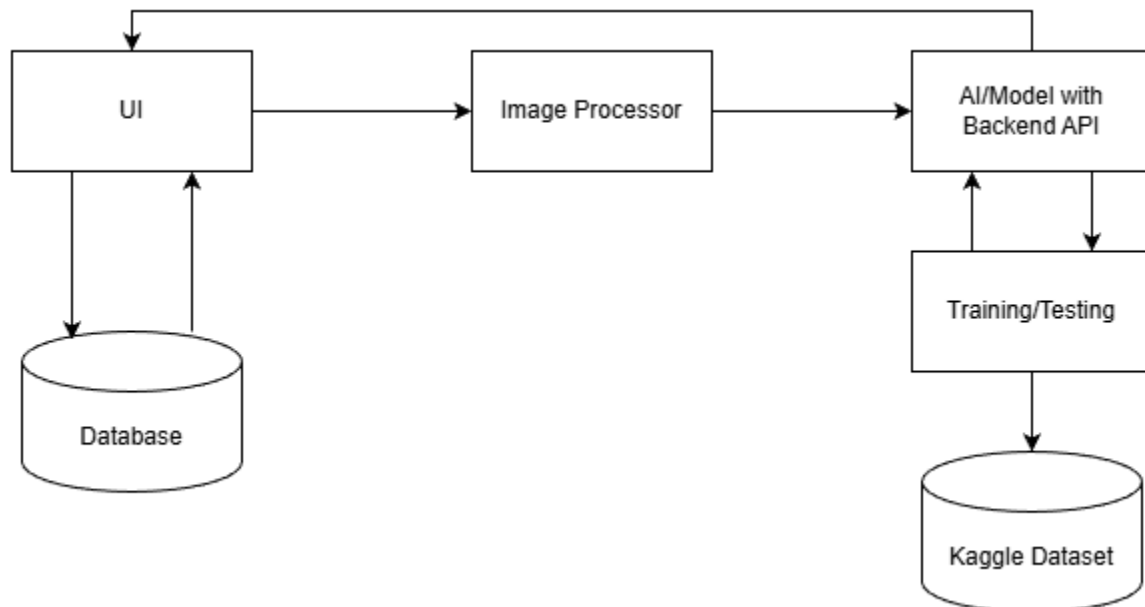
2. Version

Version 0 - Initial draft of Design and Validation & Verification documents.

3. Purpose statement for the project

The NexDerm project is a mobile application that uses machine learning and artificial intelligence technology to identify various forms of skin lesions and illnesses based on captured and uploaded images by users with preliminary diagnoses, as well as dermatological support. This document serves as an official technical description of the designed system, including component architecture, interface specifications, design, and quality assurance, as well as an extensive verification and validation (V&V) plan outlining how the developed system's functionality, performance metrics, and reliability will be validated to ensure compliance with the defined requirements and performance expectations.

4. Component diagram:



1) The user interacts with the UI to upload or capture a skin image. For authenticated users, the UI may also store or retrieve related data from the Database.

- 2) The uploaded image is sent from the UI to the Backend API (FastAPI), which receives the request and forwards the image to the Image Processor, where it is validated, resized, normalized, and prepared in a model-ready format.
- 3) The processed image is forwarded to the AI Model, which performs skin-condition classification and returns a predicted label and confidence score back to the Backend API and then to the UI for display.
- 4) Separately, the Training/Testing Module uses the Kaggle dataset to train and evaluate the model offline, and the trained model is deployed to the AI Model component used during runtime inference.
- 5) After inference, the backend may optionally trigger report generation for authenticated users.
- 6) The frontend may invoke the dermatologist locator feature using device location and Google Maps integration.

5. Relationship Between Project Components and Requirements:

User Interface (UI) / Frontend (Mobile Application)	P0 - Users must be able to upload a skin image through the application UI. P0 - Users must be able to 'Continue as Guest' (i.e., upload and get results without logging in). P0 - The application must display the classification result to the user in a clear and accessible format. P0 - Software must include basic error handling (invalid input, server errors). P0 - Users must be able to interact with the application via a mobile interface. NFR - The mobile application will use a clean, professional-looking design consistent with healthcare-related design expectations. NFR - The design should be responsive and usable through mobile devices on various screen sizes. NFR - All error messages must be clear, actionable, and non-technical, and the application must display a medical disclaimer.
Image Processor	P0 - Validates uploaded skin images for format, size, and integrity before processing. P0 - Receives images from the frontend and prepares them for AI inference. P0 - Resizes, normalizes, and converts images into the tensor format required by the model. P0 - Detects invalid or corrupted images and returns appropriate errors. NFR - Processes images within a few seconds under normal conditions. NFR - Ensures images are used only for real-time inference and not stored without consent.

Database	P1: Manages user authentication state, including secure login, registration, and session token validation. P1: Stores persistent records of user upload history, including image metadata and classification results. P2: The system should support data-retention controls consistent with the project's privacy policy. NFR: Ensures data is encrypted at rest (AES-256) and in transit; guarantees high availability for authentication services.
AI Model Subsystem	P0: Loads the trained deep learning model and performs real-time inference on preprocessed image tensors. P0: Classifies input images into supported skin disease categories or “healthy” and returns prediction results to the backend API. P0: Generates confidence scores associated with each prediction to support uncertainty handling. NFR: Ensures inference requests are processed within a few seconds under normal operating conditions. NFR: Maintains compatibility with backend inference services without requiring additional preprocessing modifications.
Training/Testing Module	P0 - A trained deep learning model must classify an uploaded skin image into one of the supported skin disease categories or “healthy.” P0 - The model must be trained using labeled dermatological image data sourced from the Kaggle Skin-DS dataset. P0 - The model must be evaluated on a held-out test dataset used exclusively for final performance assessment. P0 - The trained model must achieve minimum acceptable performance thresholds ($\geq 60\%$ accuracy, ≥ 0.50 F1-Macro, ≥ 0.70 AUC) before deployment. P0 - The training pipeline must compute standard evaluation metrics, including accuracy, F1-score, AUC, and confusion matrices. P0 - The training process must generate and save a finalized model artifact for use by the backend inference API. P0 - The training and testing workflow must support GPU-accelerated execution to enable deep learning model training. P1 - The training pipeline should generate evaluation artifacts (e.g., ROC curves and precision-recall curves) to support analysis and reporting. NFR - The training and testing process must be reproducible through consistent preprocessing, fixed label mappings, and saved model checkpoints. NFR - Training and evaluation must follow ethical data-handling practices, ensuring no user images are retained without explicit consent. NFR - The trained model must be compatible with backend inference services without requiring additional preprocessing changes.

6. Normal Behavior, API, Implementation, Undesired Behavior:

6.1 User Interface (UI) / Frontend (Mobile Application)

a. Normal Behavior: The UI allows users to select or upload a skin image, submit it for analysis, and view the returned classification result and confidence score. It presents results clearly and provides warnings when predictions are uncertain.

b. API (Inputs/Outputs)

- Input: User-selected skin image (JPEG/PNG).
- Output: Structured JSON response from the backend containing the predicted class, confidence score, and error messages if applicable.

c. Implementation: The UI is implemented as a mobile application using Expo React Native. It consists of screens for image upload, result display, authentication, and error handling. The UI communicates with the backend via REST API calls implemented in FastAPI over HTTP/HTTPS and performs basic client-side validation before submission.

d. Potential Undesired Behaviors: The UI may encounter unsupported image formats, network failures, server errors, or low-quality images. These issues may lead to failed uploads, delayed responses, or unclear results if not properly handled.

6.2 Image Processor

a. Normal Behavior: The Image Processor receives uploaded skin images from the frontend, validates them, performs preprocessing, and prepares them for AI model inference. It ensures images are consistently formatted and ready for classification before forwarding them to the model.

b. API (Inputs/Outputs)

- Input: Image file uploaded by the user (JPEG/PNG).
- Output: Preprocessed image tensor sent to the AI inference service and structured JSON responses containing either model-ready data or error messages.

c. Implementation: The Image Processor is implemented as part of the backend using Python with FastAPI. Image preprocessing includes format validation, resizing, normalization, and tensor conversion. This logic is encapsulated within preprocessing utility functions that are called by the image upload endpoint before invoking the model API.

d. Potential Undesired Behaviors: The component may encounter unsupported image formats, corrupted files, poor image quality, or oversized uploads. These cases may result in failed preprocessing, increased latency, or inaccurate model predictions if not properly handled.

6.3 Database Subsystem

a. Normal Behaviour: The Database Subsystem operates passively, responding to read and write requests initiated by the Backend API and the Mobile Frontend Client. Under normal operation, the database allows user authentication (login/registration) and stores the classification result (disease label), the calculated confidence score, and the filename and location of the processed image. This data is strictly linked to the registered user via a foreign key relationship. The system also handles queries for user history, allowing the application to retrieve past diagnostic results based on the user's ID and role.

b. Tables and Schema:

Table 1: users

Field	Type	Constraints
id	SERIAL	PRIMARY KEY
email	VARCHAR(255)	UNIQUE, NOT NULL
hashed_password	VARCHAR(255)	NOT NULL
role	VARCHAR(50)	NOT NULL, DEFAULT 'patient'
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT CURRENT_TIMESTAMP

Table 2: classifications

Field	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	FOREIGN KEY (ref users.id), ON DELETE CASCADE
image_filename	VARCHAR(255)	None
classification_result	VARCHAR(100)	None
confidence_score	DECIMAL(5, 4)	None
user_consent	BOOLEAN	NOT NULL, DEFAULT FALSE
timestamp	TIMESTAMP WITH TIME ZONE	DEFAULT CURRENT_TIMESTAMP

c. Implementation: The database is implemented using PostgreSQL with Supabase. The system stores user account information, classification results, report metadata, and related history for authenticated users. Data retention is governed by user consent and account status.

If a user requests account deletion via support email, all personally identifiable account data and associated diagnostic records are permanently deleted, unless explicitly retained under anonymized conditions for research or system improvement.

Access to data is governed by authentication and authorization checks enforced by the backend.

d. Potential Undesired Behaviors: One potential issue is inconsistency between account deletion behavior and the data retention policy. To avoid retaining user-linked records after consent is revoked, the system must ensure that deletion or anonymization policies are applied

consistently.

Additional risks include rounding of confidence values for storage and possible concurrent write conflicts, though these are mitigated through database constraints and backend-side validation.

6.4 AI Model Subsystem (with Backend API)

a. Normal Behavior: The system uses an ensemble of convolutional neural networks (DenseNet121 and ResNet50) combined with a preliminary skin-region filtering step. The neural network predicts a value for the probability of a given class of skin diseases that are supported in the application. The results produced by the model consist of the predicted class for the disease and the confidence level linked to this prediction, which is the probability score associated with the predicted output. This is then supplied to the user through the front end of the system.

b. API and Structure

The AI model subsystem consists of the following primary components and functions:

- **load_from_checkpoint(checkpoint_path):** Loads the pretrained ensemble model components from saved checkpoints, including classification layers, normalization parameters, and class-label mappings. Model parameters will then be located in the correct position (CPU/GPU) and switch to evaluation mode.
- **preprocess_image(image):** Receives the preprocessed image from the Image Processor and outputs a batch tensor ready for inference.
- **predict(image):** This function takes a PIL image as input and performs inference on the model. This function employs a softmax function to calculate the probabilities for each class after obtaining the logit values from the inference result and returns the predicted class and level of confidence for the disease classification.
- **EnsembleDiseaseClassifier Class:** This class encompasses the entire prediction process, from device management and preprocessing setup to model loading and prediction execution. The ensemble model uses a fine-tuned model convolutional neural network with its own classification head.
- **FastAPI Backend API Endpoints:** The `/predictions` endpoint receives the uploaded image from the frontend, passes it to the model service for preprocessing and inference, and returns the prediction and confidence score to the frontend.

c. Implementation: The inferences in the pipeline start when the user uploads a picture of the skin image via the frontend interface. The backend system (FastAPI) receives the request at the `/predictions` endpoint and submits the image to the model, where the request is processed by the instance of the `EnsembleDiseaseClassifier` class. First, the model is loaded using the "load_from_checkpoint" function after the model has been trained initially. As soon as the image comes in, the "preprocess_image" function prepares the image in a tensor format for the neural network.

Finally, the tensor is fed into the ensemble model using the predict function. The model returns the logits, which are mapped to probability values using the softmax activation function. The argument with the highest probability value is chosen as the prediction label for the disease. Finally, the prediction label and confidence value are collected as a structured response and sent to the frontend for display by the backend.

d. Potential Undesired Behaviors: One of them is false positive identification, where the model identifies a healthy skin image as diseased. The other key risk is that of false negatives, or when the model fails in predicting the presence of a skin condition. Of these two, false negatives are more serious since they will delay medical consultation and treatment.

The reasons for the poor performance of the model are also low-quality images, occlusions, image blur, or low and inconsistent lighting conditions in the photos uploaded. Also, differences between images from the real world and the distribution in the training dataset will result in lower generalization accuracy. System-level issues like increased inference latency, high server load, or temporary backend downtime may cause slower responses or unavailability of prediction services.

6.5 Training/Testing Module

a. Normal Behavior: The AI Model Training and Testing Module operates offline during development and model update phases to produce a trained deep learning model for skin condition classification. It loads the Skin-DS dataset, applies standardized image preprocessing, and trains a deep learning model (DenseNet121 and ResNet50 ensemble) using transfer learning. Model performance is monitored on a validation dataset during training, and final evaluation is performed on a held-out test dataset. The best-performing model is saved and deployed to the AI Inference Service. This module does not handle live user requests and does not interact directly with end users.

b. API (Inputs/Outputs) :

- **Input:** Skin-DS dataset directory structured into train/, val/, and test/ folders with class-labeled subdirectories (image files), plus training configuration parameters (e.g., batch size, epochs, learning rate, checkpoint path).
- **Output:** Trained DenseNet121 and ResNet50 model checkpoints (best/last weights) and evaluation artifacts, including a classification report, confusion matrix, ROC curves, and precision-recall curves, saved for later analysis and reporting.

c. Implementation :

- **Dataset Preparation** - The model is trained and evaluated using the Skin-DS dataset from Kaggle, organized into separate training, validation, and test splits with class-specific directories to ensure proper data separation.
- **Image Preprocessing** - All images are resized to 224×224 pixels and normalized using ImageNet statistics to ensure compatibility with the pretrained DenseNet121 and ResNet50 architecture.
- **Model Architecture** - A DenseNet121 and a ResNet50 model initialized with ImageNet weights are used, with frozen feature extraction layers and a modified classifier head to match the number of skin disease classes.
- **Training Procedure**- The model is trained using mini-batch gradient descent with cross-entropy loss and the Adam optimizer over multiple epochs, utilizing GPU acceleration when available.
- **Validation During Training** - Model performance is evaluated after each epoch on a validation dataset by computing validation loss and accuracy to monitor generalization and detect overfitting.

- **Model Checkpointing** - Model checkpoints are saved during training, with the best-performing model selected based on the lowest validation loss and preserved for deployment and testing.

d. Potential Undesired Behaviors: The model may overfit to dominant classes in the dataset, resulting in reduced performance on underrepresented conditions. Visually similar skin diseases may also be misclassified due to overlapping visual features in the image data. Training may be impacted by hardware limitations such as insufficient GPU memory or by errors in dataset organization and preprocessing. These risks are mitigated through validation monitoring, consistent preprocessing, proper dataset management, and systematic evaluation prior to deployment.

6.6 Dermatologist Locator Module

a. Normal Behavior: The Dermatologist Locator module uses the user's geolocation to identify nearby dermatologists and displays them in a map-based interface.

b. API (Inputs/Outputs):

Input: User latitude/longitude.

Output: A list of nearby dermatologists including clinic name, address.

c. Implementation: The frontend integrates with Google Maps Platform APIs to retrieve nearby dermatology clinics using the user's device location. The results are displayed in both map and list formats.

d. Potential Undesired Behaviors: Missing location permissions, inaccurate GPS data, or API failures may prevent results from being displayed correctly.

6.7 Report Generation Module

a. Normal Behavior: After a prediction is generated for an authenticated user, the system creates a structured report containing the classification result, confidence score, and generated explanatory content, and makes it available for later viewing or download as a PDF.

b. API (Inputs/Outputs):

Input: Prediction label, confidence score, user/report metadata, and generated report text.

Output: PDF report file and stored report metadata accessible through the backend.

c. Implementation: The backend generates report text through the report-generation service, formats the result into a PDF document, stores report metadata, and returns a downloadable or retrievable report reference to the frontend.

d. Potential Undesired Behaviors: Report generation failure, delayed background processing, missing metadata, storage failure, or invalid download links may prevent the user from accessing the final PDF.

7. Details on User Interface:

User Interface Design: The NexDerm User Interface is a clean, minimalistic mobile-first design suitable for the healthcare industry. Clarity, simplicity, and user confidence are the UI's primary goals. Therefore, users without any technical or medical knowledge will be able to easily interact with the application.

Layout and Navigation: The application follows a task-oriented structure: image upload → analysis → results. Logged-in users will see additional screens through a navigation structure that displays: report history and account settings. Guest users can upload images and view results for the current session only.

Image Upload Screen: The upload option can be accomplished by selecting one of your saved images or taking an image of yourself with your device's camera. Directions for selecting an image are available above the center of the image upload area. After uploading an image, a preview of the uploaded image will be displayed for confirmation prior to submission.

Results Screen: The results screen displays the predicted skin condition, confidence score, and an uncertainty notice when applicable. It also provides actions for generating a PDF report (only for logged in users) and accessing the dermatologist locator feature.

Report History Screen: The report history screen displays a chronological list of previous reports for authenticated users, including prediction summaries, timestamps, and access to detailed report content or downloadable reports.

Dermatologist Locator: This feature will allow users to click on the results screen and find local dermatologists on a map using their location information. The feature will show the distance to local dermatologists as well as basic information about the clinic so users can make informed decisions on their next steps.

Visual Design (*COLORS AND FONT*): Light and neutral colours provide the base of the platform, while some areas of the platform include accent colours (blue and green) to represent a clinical, trustworthy look. The default font used throughout the platform is San Francisco (used on iOS devices). The font has been designed for ease of reading with consistent spacing and the hierarchy of primary and secondary content clearly defined.

User Interaction and Feedback: Users receive instantaneous visual feedback upon performing any actions (e.g., buttons, navigation elements, and forms). Load indicators will be displayed while waiting on an analysis or report to generate. Error messages are brief, written in layman's terms, and provide actionable steps for how to fix issues encountered.

Design Reference: The user interface was designed and then written in code; no external design tools were used during initial development; Figma was later used as a reference for documenting the UI design in the process of creating the user interface.. The design, component locations, and flow of interaction within the design evolved as a result of many attempts and errors to ensure the most usable experience possible, based on project needs and requirements. The user interface that was developed and published is the definitive reference for the designs and behaviours of the user interface.

Figma:

<https://www.figma.com/design/WjcLuijD6DRv9w6yn9fIZt/NexDerm?node-id=0-1&t=BbzaIP5TxUUne2CY-1>

Validation and Verification

1. User Interface (UI) / Frontend (Mobile Application)

1.1.1 Unit Tests (Testing Methodology): The unit tests are designed to verify that a user can upload a skin image using the application interface, continue as a guest user without having to log in, and receive the classification results. Additionally, the unit tests verify that the result and confidence score are presented clearly, that low-confidence predictions will trigger an uncertainty message, and that any invalid input, server errors, timeouts, or network failures will display a clear, user-friendly, non-technical error message. The responsive nature of the application is tested by simulating uploads from various mobile device sizes to verify that there are no layout or text issues.

1.1.2 Tools/Methodology: Testing is performed using Jest and the React Native Testing Library for component- and screen-level tests. Backend API responses are mocked to simulate successful responses, errors, and timeouts, and snapshot tests are used for key screens, including image upload, results, and error states.

1.2.1 Performance Tests and Metrics: Performance testing of the application measures the perceived latency between the time an image is submitted and the time the results are shown. The latency must be less than a few seconds on average for images under normal network conditions. In addition to measuring latency, performance testing also measures the application's responsiveness to UI changes used in images submitted for classification. The responsiveness of the UI is tested throughout the upload process and while in navigation, and no significant UI freezing or UI lagging will occur.

1.2.2 Measurement Methodology: We are measuring performance data using React Native Performance Monitor or Flipper to track frame rates and behavior of the JavaScript threads. Timestamps recorded throughout the lifetime of an API request can help measure the overall latency of the time taken for the app to send and receive data. To get an average view of how well your application performs, create a performance test using a minimum of two device profiles.

2 Image Processor

2.1.1 Unit Tests: The Image Processor is responsible for validating, preprocessing, and preparing user-uploaded skin images for inference by the AI model. The backend exposes REST API endpoints, and testing will be conducted using Pytest.

2.1.2 Image Upload and Validation: Tests validate supported file formats (JPEG, PNG) and verify that users cannot submit unsupported or corrupt image files. Tests also verify the handling of missing image files/required metadata, as well as the enforcing of file size limits. A 200 status code is returned for all valid uploads, and a 400 status code will be returned for all invalid submissions, with an appropriate and explicit message.

2.1.3 Image Preprocessing: Tests validate that images can be resized correctly according to specifications, and that normalizations are performed properly, with accurate channel conversions, with no accidental distortions or loss of data, and that the outcome tensor is generated in the same shape and datatype as the expected output. Test images are created to help compare preprocessing results with known expected outcomes.

2.1.4 Model Input Preparation and Output Handling: The tests ensure that the transferred images are preprocessed images sent to the inference endpoint. The tests also confirm that JSON format is used for predicted results and that each result contains both a confidence score and a label. The tests will catch any instances of errors when invalid or incomplete results are returned from the model.

2.2 Performance Tests and Metrics: Latency is the primary performance metric for the image processor. To perform performance testing, load testing tools will be used to simulate a scenario where more than one user can perform thorough image uploads concurrently. Through this simulation testing, metrics that will be captured will include average processing time for each image submitted, how the system reacts when concurrency increases, and the percentage of errors when processing a large volume of requests. The system will be compliant with the specified performance metrics when all image preprocessing, validation of input images, and preparation of responses are completed within **5-10 seconds** for a typical volume.

3 Database Subsystem

3.1 Unit Tests: Database integrity is verified through a suite of automated tests integrated into the backend's CI/CD pipeline using Pytest. These tests ensure that the PostgreSQL schema correctly enforces constraints, such as unique user emails and valid foreign key relationships. Specific test cases validate the user account deletion behavior, confirming that classification data associated with deleted users is removed or anonymized in accordance with the system's data retention policy. Additionally, security tests simulate database access from different user contexts. All tests execute within transaction rollbacks to ensure a clean database state is maintained between runs.

3.2 Performance Test and Metrics: Database performance testing ensures the system stays responsive as data grows, especially for history retrieval and saving new results. Metrics include read latency (SELECT history), write latency (INSERT into classifications), and index utilization (queries use B-tree indexes, not full scans). Acceptance criteria will be that target history query latency is under 100 ms, concurrent writes without lock/timeouts, and row-level security adds minimal overhead.

4 AI Model Subsystem (with Backend API)

4.1 Unit Tests: Unit testing of a system is carried out to check the accuracy and robustness of the pipeline of the dermatology image classification system and the backend API inference flow. This testing is carried out for the correctness of the loading of the designed and implemented ensemble model pipeline from the checkpoint files, the correctness of the implementation of image preprocessing methods such as resizing, normalization, and conversion to the RGB color space of the image, and the correctness of the returns related to prediction results regarding the class and confidence levels. Error testing related to the absence of model files, invalid image input, and API request handling is also carried out.

4.2.1 Performance Tests and Metrics: Due to the potential class imbalance problem that might exist in dermatological image classification tasks, the accuracy of the models is evaluated using precision metrics, recall/sensitivity measures, F1 scores, and confusion matrices. In an attempt to test the efficiency of models developed in the thesis work, they are validated using a separate test dataset independent of a training dataset by using proper evaluation libraries for calculating metrics. The efficiency of models during inference and the backend API response time are measured using mean prediction times per image.

4.2.2 Acceptance Criteria: The model should be able to handle F1 score metrics on different classes of disease, with a preference for recall to minimize false negative results. The average inference time should be less than five seconds when subjected to normal loading conditions, and the AI system should not produce any runtime errors.

4.2.3 Real-World Photo Generalization Testing: In addition to evaluation on the held-out Kaggle Skin-DS test set, the model will be evaluated on a small dataset of real-world images collected independently by the team from internet sources. These images reflect varied conditions such as uneven lighting, blur, distance variation, and background noise. The purpose of this testing is to assess the gap between curated dataset performance and real deployment conditions. Performance on this set will be analyzed qualitatively and quantitatively where possible, with particular attention to false negatives, uncertain predictions, and failures caused by poor image quality.

If available, evaluation on an additional external dermatology dataset may also be performed to further assess generalization.

5 Training/Testing Module

5.5.1 Overview: The Verification and Validation (V&V) process of a skin condition classification model evaluates that it is correct, robust, and reliable because it is developed in a medical context, and that there is class imbalance in the Skin-DS dataset. Therefore, while the model's overall accuracy will be assessed using this one metric of overall assessment, other metrics will also be used to evaluate the model's overall and class-wise performance.

5.5.2 Dataset Split and Overfitting Prevention: Tests exist to validate that the splitting of data sets (into training, validation, and test sets) and the training workflow do not lead to either overfitting or data leakage during training. A test verifies that the data was properly separated

into the three sets (training, validation, and test), that only training data is used to optimize model parameters, that validation metrics are computed after each epoch, that the best model is chosen as the one with the lowest validation loss, and that the test set is only used for final evaluation. A test passes if the model training and selection occur without any influence from the test data.

5.5.3 Confusion Matrix Analysis: In the Class-wise Performance Test, tests generate confusion matrices (count-based and row-normalized), verify that predicted labels are aligned correctly with their corresponding true label, verify that the diagonal values of the confusion matrix represent correctly classified samples for each class, and verify that there are consistent patterns of misclassification for a class. For a confusion matrix to pass, the way in which the model makes predictions must be properly reflected in the matrices.

5.5.4 - 5.5.6 Precision, Recall, and F1-Score: Evaluating precision, recall, and the F1 score allows a thorough evaluation of a model's classification performance across all classes. When performing evaluations on a per-class basis, tests will validate precision and recall metrics using a one-versus-all method ("OAR") to ensure accurate calculation of true positive, false positive, and false negative counts and macro-averaged metric computations. F1-score validation is also performed through the calculation of the harmonic mean of these three metrics and the calculation of both macro and weighted F1-scores. All of the above metrics will be included in the final classification report (e.g., precision, recall, F1-score, and macro-average). When all of these metrics have been accurately calculated and reflect the true performance of the model (both at the class level and overall), this satisfies the test requirements.

5.5.7 Precision-Recall Curve: The evaluation process includes to generate precision-recall curves using predicted probabilities and perform one-versus-rest comparison to compute precision-recall curves for each class separately, derive average precision (AP) scores, and save these precision-recall curves so they can be used in future evaluations. Any testing of the precision-recall curves and average precision scoring would be deemed successful if they were generated and saved properly.

5.5.8 ROC Curve and AUC: The steps taken for ROC analysis included generating one-versus-rest ROC curves; validating the True Positive Rate (TPR) and False Positive Rate (FPR) from the ROC curves; deriving Area Under Curve (AUC) values for each class along with the Macro Averaged AUC Value; and reporting Macro Average AUC Values. Any testing would be deemed successful if the ROC curves and AUC values were above the minimum expected level of performance based on random guessing.

5.5.9 Model Performance Validation: Model training completes with one last validation step called final validation, which ensures that the trained model has met all performance criteria and is ready for production deployment. Model parameters will not update during the testing phase; instead. If all metrics (macro-averaged F1 score and area under the curve) are acceptable values, and all evaluation artifacts are produced correctly, then final validation is successful. The required final outputs for inspections and reports have been generated.